

---

**Faculty of Engineering**  
Department of Communications & Electronics  
*Microprocessors & Interfacing — Spring 2026*

---

P R O J E C T   R E P O R T

# Letter Recall Game

*8086 Assembly Language Implementation*  
EMU8086 Emulator · Random Sequence Edition

---

**Prepared by:**

|                                         |                      |
|-----------------------------------------|----------------------|
| Name: عمر عادل محمد عبد العزيز          | Student ID: 23010609 |
| Name: عبد الله يحيى السيد علي           | Student ID: 23011751 |
| Name: عبد الرحمن شكري محمد صالح         | Student ID: 23010528 |
| Name: عبد الرحمن ابراهيم عبد الحميد حسن | Student ID: 23010505 |

## 1. Introduction

---

This report documents the design, implementation, and testing of the Letter Recall Game, a memory-training application developed entirely in 8086 Assembly Language and executed on the EMU8086 software emulator. The project was undertaken as part of the Microprocessors & Interfacing course and demonstrates practical application of low-level programming constructs including memory addressing, interrupt service routines, loop control, and I/O operations.

Assembly language programming is one of the most fundamental disciplines in computer engineering. Unlike high-level languages, Assembly requires the programmer to manage every CPU register, memory location, and hardware interrupt explicitly. The Letter Recall Game serves as an effective and engaging vehicle for demonstrating these concepts while producing a fully functional, interactive application that runs directly on 8086-compatible hardware.

The game challenges the player to memorize a sequence of randomly generated uppercase letters shown briefly on the screen and then correctly re-enter them from memory. Three progressively harder levels are provided, and a configurable difficulty system allows the player to adjust the duration for which the letters remain visible.

## 2. Project Objective

---

The primary objectives of this project are as follows:

- Apply 8086 Assembly Language programming to build a complete, interactive application.
- Demonstrate proficiency with DOS and BIOS interrupt calls (INT 21H, INT 10H, INT 1AH).
- Implement a pseudo-random number generator using BIOS timer values (LCG algorithm).
- Design a multi-level game structure using structured procedures and clear control flow.
- Provide configurable difficulty through a delay mechanism that adjusts memorization time.
- Handle user input with case-insensitive validation and graceful retry logic.
- Produce readable, well-commented, maintainable assembly source code.

### 3. Game Description

---

The Letter Recall Game is a single-player, text-based memory game. Upon launch, the program displays a welcome banner and prompts the player to choose a difficulty level. After confirmation, the game progresses through three timed levels of increasing challenge:

| Level | Letters | Description                                         |
|-------|---------|-----------------------------------------------------|
| 1     | 3       | Introductory round — three random uppercase letters |
| 2     | 5       | Intermediate round — five random uppercase letters  |
| 3     | 7       | Advanced round — seven random uppercase letters     |

During each level the program:

1. Generates a sequence of random uppercase letters using a hardware-seeded LCG algorithm.
2. Displays the sequence on screen for a duration determined by the selected difficulty.
3. Clears the screen to hide the letters, prompting the player to recall and type them.
4. Compares each entered character against the stored sequence in order.
5. Awards a CORRECT result and advances to the next level, or shows the WRONG answer with a retry option.

If the player successfully recalls all three levels in sequence, a congratulatory YOU WIN message is displayed. Choosing N on the retry prompt at any level ends the game with a Game Over message.

### 4. Features

---

- Three progressive levels with 3, 5, and 7 random letters respectively.
- Hardware-seeded pseudo-random letter generation using the BIOS timer (INT 1AH) and a Linear Congruential Generator (LCG).
- Three selectable difficulty modes (Easy, Medium, Hard) that control the memorization window.
- Automatic uppercase normalization of player input (accepts both upper and lower case).
- Clear screen between phases to prevent visual cues from aiding recall.
- Retry option presented after an incorrect answer without regenerating the sequence.
- Distinct visual feedback for correct and incorrect answers using ASCII-art bordered messages.
- Designed with compatibility notes for the physical MTS-86C laboratory kit.
- Modular procedure-based architecture for readability and maintainability.

## 5. User Interface

---

The game interface is entirely text-based, running within an 80-column × 25-row character terminal as emulated by EMU8086. The interface is organized into distinct screens that are cleared and redrawn at each game phase transition, ensuring a clean, uncluttered experience.

### 5.1 Welcome Screen

The welcome screen presents the game title inside an ASCII-art border, briefly explains the game concept, and immediately prompts the player to select a difficulty level before any gameplay begins.

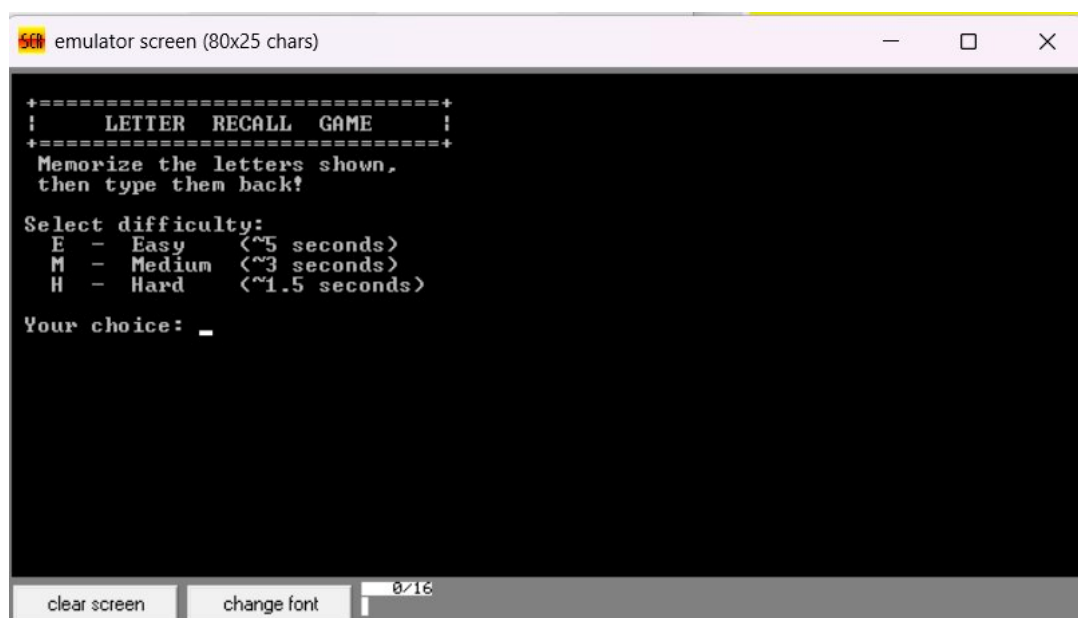


Figure 1 — Welcome screen and difficulty selection prompt as displayed in EMU8086

### 5.2 Memorization Screen

Once the difficulty is selected and the player presses any key, the program enters the active level. The current level number is announced, followed by the randomly generated letter sequence preceded by the label Memorize:. The letters remain visible for the difficulty-controlled duration, then the screen is cleared.

### 5.3 Recall Screen

After the screen is cleared the player sees only the prompt Now type the letters: followed by a cursor. Characters are echoed as the player types, each separated by a space. No backspace correction is possible, matching the physical kit behaviour.

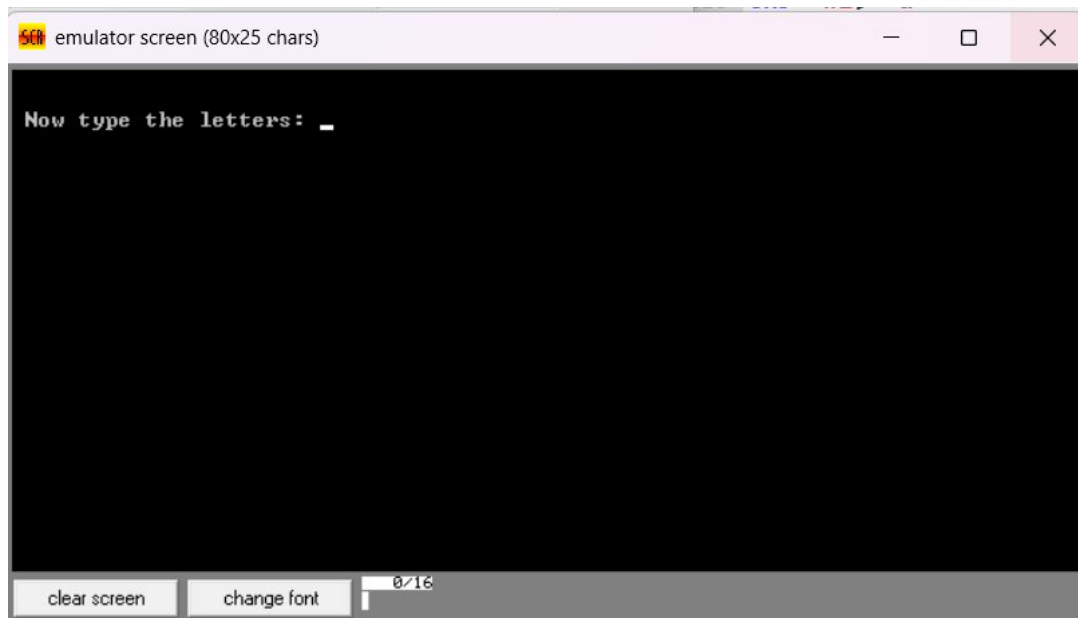


Figure 2 — Recall screen: player types letters after the memorization window closes

### 5.4 Result Screens

The result is displayed immediately after the last character is entered.

- Correct answer: A bordered CORRECT! Great memory! banner is shown.
- Wrong answer: The original sequence is revealed alongside the player's attempt, and a Y/N retry prompt is displayed.

## 6. Difficulty Modes

---

The difficulty system is implemented through the `delay_outer` variable, which governs the outer loop count of the `DELAY_LONG` subroutine. A higher value produces a longer pause, giving the player more time to memorize the sequence. The table below summarizes the mapping:

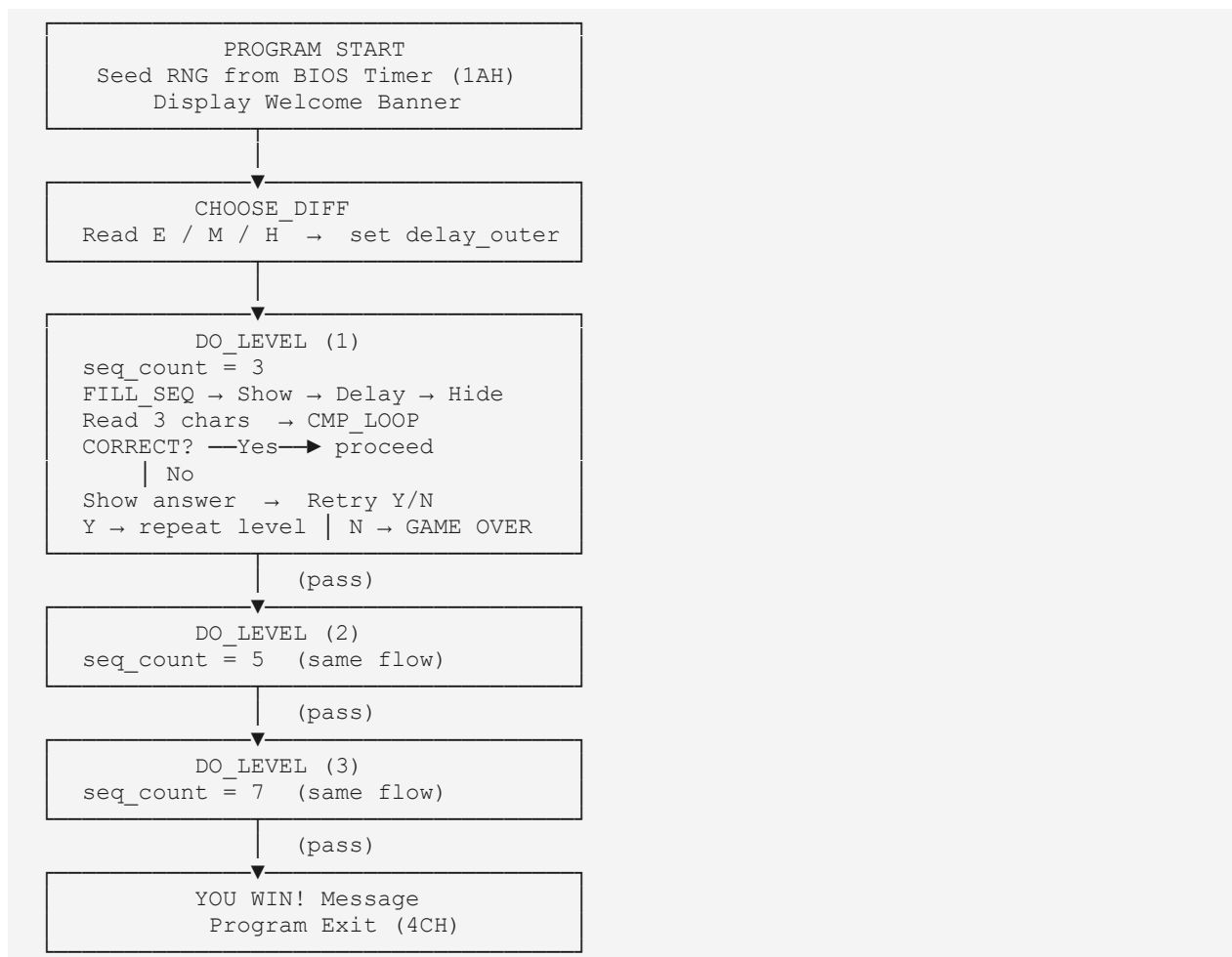
| Key | Mode   | Visible Duration | <code>delay_outer</code> |
|-----|--------|------------------|--------------------------|
| E   | Easy   | ~5 seconds       | 05H                      |
| M   | Medium | ~3 seconds       | 03H                      |
| H   | Hard   | ~1.5 seconds     | 01H                      |

The `DELAY_FACTOR` constant (set to `01H` for EMU8086) provides a scaling coefficient that can be increased (e.g., to `08H–10H`) to calibrate the delay for the higher clock speed of the physical MTS-86C laboratory kit, ensuring the same subjective memorization window regardless of the target hardware.

## 7. Program Flowchart

---

The overall program execution follows a strictly sequential, top-down flow governed by three main phases: initialization, difficulty selection, and the three-level game loop. The flowchart below describes the complete execution path at a high level.



## 8. Code Explanation

---

The source code is organized into seven discrete procedures, each encapsulating a single well-defined responsibility. The following subsections explain each procedure in detail.

## 8.1 MAIN Procedure

MAIN is the program entry point and orchestrator. It begins by initializing the DS segment register, then seeds the random number generator using INT 1AH (BIOS tick count). The XOR of the four timer bytes (DL, DH, CL) provides an unpredictable 8-bit seed that ensures different sequences each game session. After clearing the screen and printing the welcome banner, MAIN calls CHOOSE\_DIFF, waits for a keypress, and then invokes DO\_LEVEL three times in sequence. Return value AX = 0 from any level triggers a jump to SHOW\_GAMEOVER; AX = 1 allows progression. Upon completing all three levels, the congratulatory message is printed and the program exits cleanly via INT 21H / AH=4CH.

```
; Seed the RNG once at startup using BIOS ticks
MOV  AH, 00H
INT  1AH          ; CX:DX = ticks since midnight
MOV  AL, DL
XOR  AL, DH
XOR  AL, CL
MOV  rnd_seed, AL ; store initial seed
```

## 8.2 CHOOSE\_DIFF Procedure

CHOOSE\_DIFF presents the three-option difficulty menu and blocks on INT 21H / AH=08H (read character without echo). The pressed key is manually echoed via AH=02H. A bitwise OR with 20H normalises the key to lowercase, enabling case-insensitive comparison. An invalid key causes an error message to be printed and loops back to re-read. On a valid match, the corresponding delay\_outer constant is written to memory and the procedure returns.

```
OR   AL, 20H      ; force lowercase
CMP  AL, 'e'
JE   DIFF_EASY
CMP  AL, 'm'
JE   DIFF_MED
CMP  AL, 'h'
JE   DIFF_HARD
; fall-through -> invalid -> retry
```

## 8.3 GET\_RANDOM\_LETTER Procedure

This is the pseudo-random letter generator. The algorithm is a Linear Congruential Generator (LCG) seeded and mixed with live BIOS timer bytes on every call, preventing repeatable patterns even within a single game session. The steps are:

6. Read BIOS timer via INT 1AH -> CX:DX.
7. XOR DL, DH, CL, and rnd\_seed into AL.
8. LCG transform:  $AX = 109 \times AL + 11$ , then XOR with DL.
9. Reduce modulo 26 using DIV BL (BL = 26); remainder -> AH.
10. Add ASCII 'A' (41H) to map 0-25 to letters A-Z.
11. Write result back to rnd\_seed for chaining.

```
MOV  AL, 6DH      ; 109 decimal (LCG multiplier)
MUL  BL           ; AX = 109 * seed
ADD  AL, 0BH      ; + 11 (LCG increment)
XOR  AL, DL       ; mix with timer low byte
XOR  AH, AH
MOV  BL, 26
DIV  BL           ; AH = remainder 0..25
```

```
MOV AL, AH
ADD AL, 'A'      ; map to A..Z
```

## 8.4 FILL\_SEQ Procedure

FILL\_SEQ iterates seq\_count times, calling GET\_RANDOM\_LETTER on each iteration and storing each result consecutively into seq\_buf using DI as a byte pointer. CX is loaded with seq\_count and decremented by the LOOP instruction. All modified registers (AX, CX, DI) are saved and restored with PUSH/POP to preserve the caller's context.

## 8.5 DO\_LEVEL Procedure

DO\_LEVEL is the main game-logic procedure. It begins by calling FILL\_SEQ to populate seq\_buf, then enters the RETRY\_LVL loop which can be re-entered if the player elects to retry. The procedure:

- Clears the screen and announces the level number by adding cur\_lvl to the ASCII digit '0'.
- Prints the sequence character by character (SHOW\_LOOP) followed by a trailing space.
- Calls DELAY\_LONG to hold the sequence on screen.
- Clears the screen and prompts the player.
- Uses READ\_LOOP (controlled by read\_ctr in memory, not CX) to read exactly seq\_count characters, normalizing each to uppercase.
- Compares seq\_buf and user\_buf byte-by-byte in CMP\_LOOP; any mismatch branches to WRONG\_ANSWER.
- On correct: prints success message, calls DELAY\_SHORT, and returns AX = 1.
- On wrong: reveals the answer and prompts Y/N retry; Y re-enters RETRY\_LVL, N returns AX = 0.

Note on read\_ctr: INT 21H AH=02H (character output used inside READ\_LOOP for echoing) can corrupt CX in certain EMU8086 versions. A memory byte (read\_ctr) is used instead of CX to avoid this hazard.

## 8.6 CLR\_SCR Procedure

CLR\_SCR uses BIOS INT 10H / AH=06H to scroll the entire 80x25 screen to blank (fill with attribute 07H = normal white-on-black), then repositions the cursor to row 0, column 0 via INT 10H / AH=02H. The kit-compatibility note in the source shows an alternative 25-newline implementation for environments where INT 10H is unavailable.

```
MOV AX, 0600H      ; AH=06 scroll up, AL=0 full window
MOV BH, 07H        ; fill attribute: normal
XOR CX, CX         ; top-left (0,0)
MOV DX, 184FH      ; bottom-right (24, 79)
INT 10H
; reposition cursor to (0,0)
MOV AH, 02H
XOR BX, BX
XOR DX, DX
INT 10H
```

## 8.7 DELAY\_LONG and DELAY\_SHORT Procedures

Both delay routines implement busy-wait loops calibrated to the EMU8086 execution speed. DELAY\_LONG uses a two-level nested loop: the outer loop count equals delay\_outer x DELAY\_FACTOR; the inner loop counts down from 0FFH (255) each pass. DELAY\_SHORT is a fixed-duration pause (approximately 0.5 s) used after a correct answer to allow the player to read the result before screen transition. Modifying DELAY\_FACTOR scales both timings linearly for faster physical hardware.

## 9. Screenshots and Results

---

The following screenshots were captured from the EMU8086 emulator during test runs of the game across all three levels and both outcome types.

### 9.1 Difficulty Selection — Input Screen

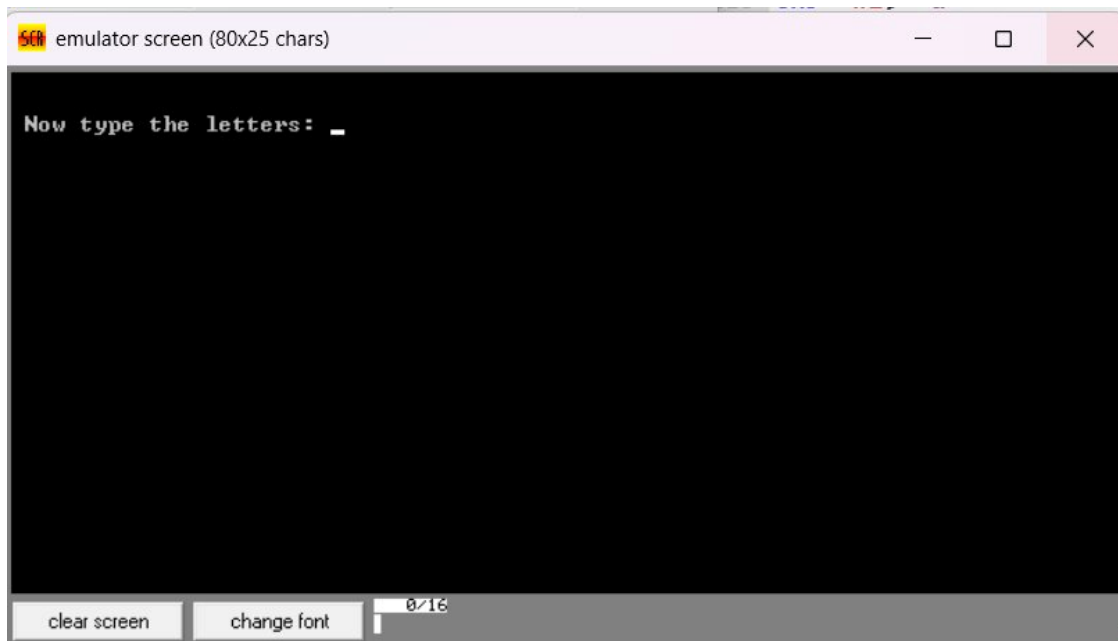


Figure 3 — Program launch: welcome banner and difficulty selection menu

### 9.2 Level 1 Result — Correct Answer

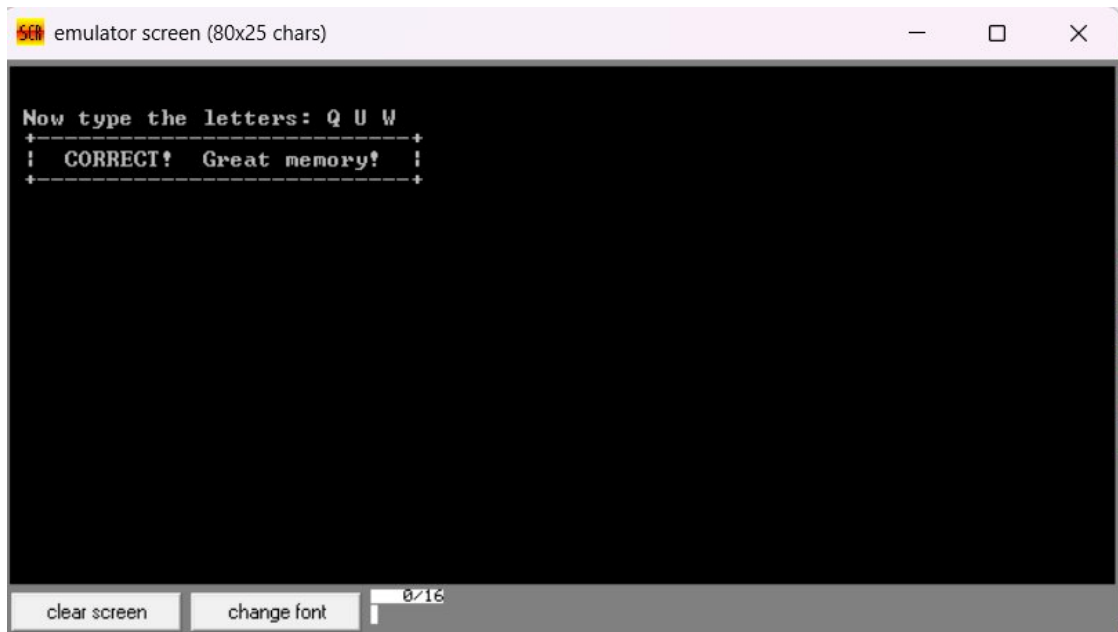


Figure 4 — Level 1 (3 letters): player recalled Q U W correctly

### 9.3 Level 2 Result — Correct Answer

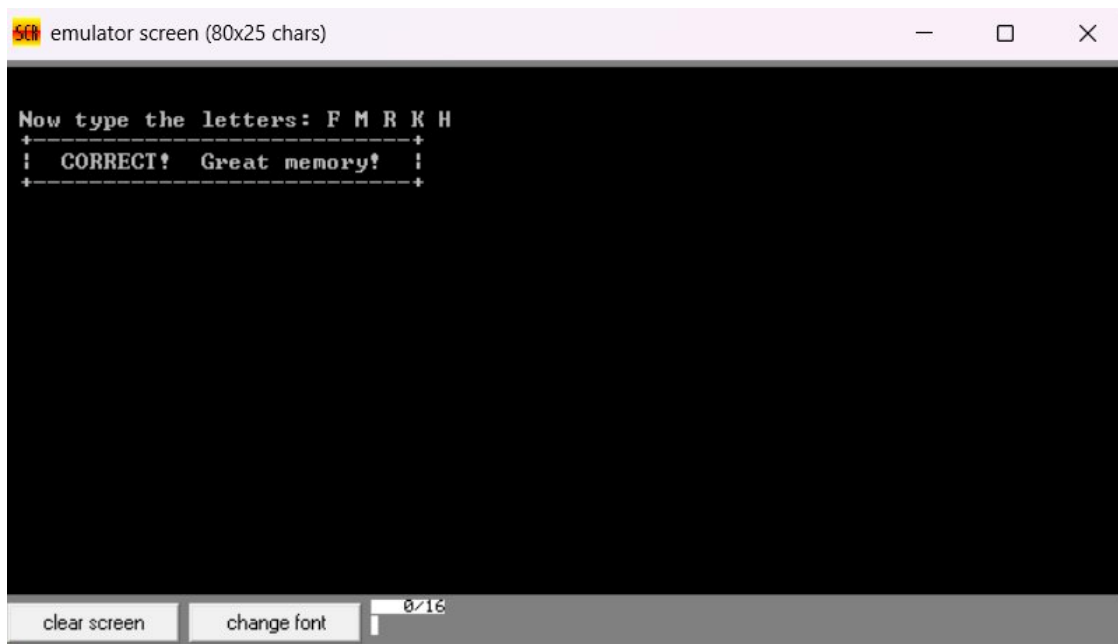


Figure 5 — Level 2 (5 letters): player recalled F M R K H correctly

### 9.4 Level 3 Result — Wrong Answer with Retry

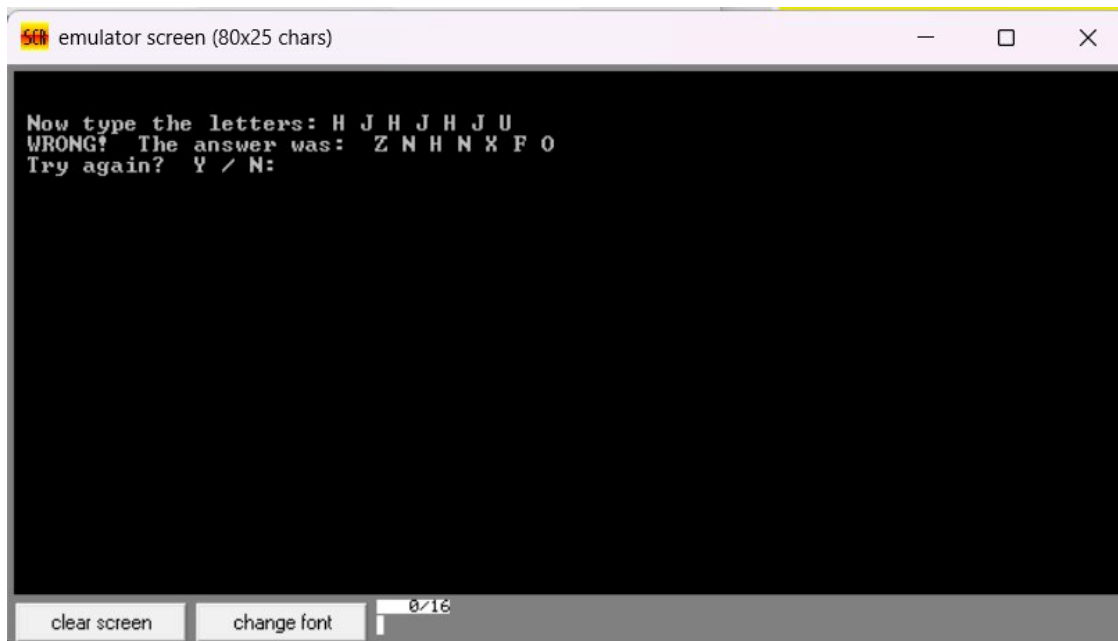


Figure 6 — Level 3 (7 letters): player answered incorrectly; correct sequence revealed with retry prompt

## 9.5 Active Gameplay — Recall Phase

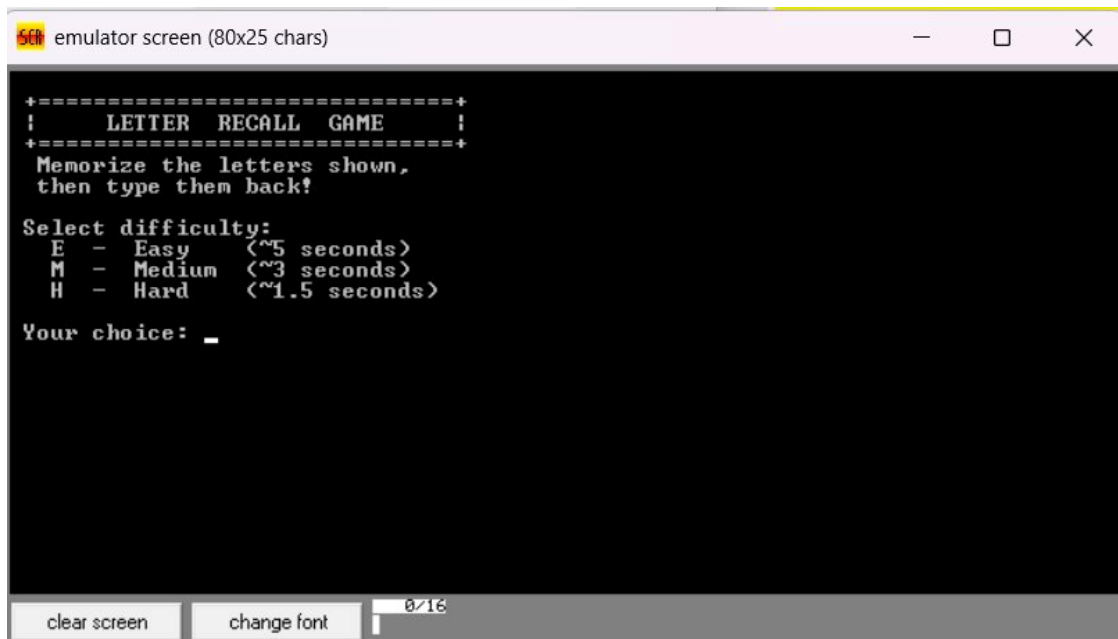


Figure 7 — Recall phase: cursor awaits player input after letters are hidden

## 10. Challenges Faced

---

Developing a complete interactive game in 8086 Assembly Language presented several non-trivial engineering challenges:

### 10.1 CX Register Corruption in Read Loop

The most significant run-time defect encountered was CX corruption caused by INT 21H / AH=02H (character output) inside the READ\_LOOP. In certain EMU8086 versions, output interrupts modify CX, silently breaking the LOOP counter mid-input. The solution was to replace the CX-based LOOP with a dedicated memory byte (read\_ctr) that is explicitly decremented and tested with DEC / JNZ, completely decoupling the loop counter from any register that an interrupt could corrupt.

### 10.2 Pseudo-Random Generation Without a Standard Library

Assembly provides no built-in random function. A custom LCG algorithm seeded from the BIOS tick counter was designed and tuned. XOR-mixing multiple timer bytes (DL, DH, CL) with a rolling rnd\_seed variable before and after the LCG transform prevents short repeat cycles and ensures adequate entropy for the game's purposes.

### 10.3 Difficulty-Calibrated Delays

Accurately timing the memorization window without an OS timer or sleep function required careful calibration of nested busy-wait loops. The DELAY\_FACTOR constant abstracts the hardware speed, allowing the same source code to target both the software emulator (DELAY\_FACTOR = 01H) and the physical 10 MHz MTS-86C kit (DELAY\_FACTOR ≈ 08H–10H).

### 10.4 Hardware vs. Emulator Compatibility

Several BIOS services available in EMU8086 (INT 10H screen clear, INT 1AH timer) may be absent or behave differently on the physical kit. All platform-specific sections are isolated in their respective procedures with clearly marked KIT NOTE comments providing tested alternative implementations, following good portability practice.

### 10.5 Input Normalization

Accepting both upper and lower case input without the standard C-library tolower() function required a manual range check (CMP 'a' / JB / CMP 'z' / JA) combined with a subtraction (SUB AL, 20H) to convert lowercase letters to uppercase in-place.

## 11. Conclusion

---

The Letter Recall Game successfully demonstrates the practical application of 8086 Assembly Language in building a complete, interactive, user-facing application. All stated project objectives were met: three progressive levels with increasing sequence length are implemented, a hardware-seeded LCG provides non-repeating random sequences, a three-tier difficulty system with calibrated delays gives players control over challenge level, and robust input handling with retry logic provides a polished user experience.

The project reinforced several key low-level programming competencies: direct hardware interrupt usage (INT 10H, INT 21H, INT 1AH), careful register management and stack discipline, memory-resident variables as CX substitutes to avoid interrupt side-effects, and modular procedure design that scales to physical embedded hardware.

Future enhancements could include a scoring system tracking the number of retries, a timer-based scoring bonus, support for mixed uppercase/digit sequences, or a persistent high-score table stored in a DOS file using INT 21H file services.

This project demonstrated that even constrained environments with no operating system, no standard library, and a 16-bit register set are fully capable of hosting engaging, interactive software — a testament to the enduring relevance of low-level programming in computer engineering education.